

DESARROLLO ORIENTADO A OBJETOS [DOPO]

2026-2

Ciclo 3

Materia: Desarrollo orientado a objetos

Grupo: 7

Integrantes

Jhon Eduard Duran Ceballos

ID: 1000100325

Correo: jhon.duran-c@mail.escuelaing.edu.co

Andres Steven Oyuela Mendez

ID: 1000109048

Correo: andres.oyuela-m@mail.escuelaing.edu.co

MINI-CICLO 1: Creación dinámica (create slot machine)

Estado : Completo

Objetivo: Inicializar la máquina con **n** ruedas y símbolos aleatorios, y crear la clase respectiva.

- Crear el constructor SlotMachine(n: int) en la clase SlotMachine para inicializar aleatoriamente una máquina de n ruedas y n símbolos.
- Crear la nueva clase SlotMachineContest.

MINI-CICLO 2: Motor de solución algorítmica (solve)

Estado : Completo

Objetivo: Crear el método solve haciendo llamado al método privado y dejando esta acción como invisible.

- Crear el método solve(n: int) : int[][] en SlotMachineContest.
- Implementar la lógica para calcular y retornar la secuencia de acciones {i,j} necesarias para ganar.
- Utilizar SlotMachine estrictamente como testing tool (modo invisible), empleando únicamente los métodos SlotMachine(n), spin(wheel,steps) y distinctSymbols() para resolver el algoritmo.

MINI-CICLO 3: Simulación visual (simulate)

Estado : Completo

Objetivo: Crear el método simulate haciendo llamado al método privado y dejando esta acción visible.

- Crear el método simulate(n: int) : void en SlotMachineContest.
- Implementar la simulación gráfica de las acciones necesarias para ganar.
- Configurar el simulador para que permanezca en modo visible usando los métodos makeVisible() y makeInvisible() de SlotMachine.

MINI-CICLO 4: Pruebas de unidad y aceptación

Estado : Completo

Objetivo: Contruir los respectivos test para la máquina logrando que sean compatibles con respecto a las definiciones de los métodos anteriores.

- Construir los casos de prueba de unidad propios en la clase SlotMachineContestTest.
- Implementar mínimo dos casos de prueba de unidad compartidos en la clase SlotMachineContestCTest.
- Desarrollar dos pruebas de aceptación preparadas para la presentación final.

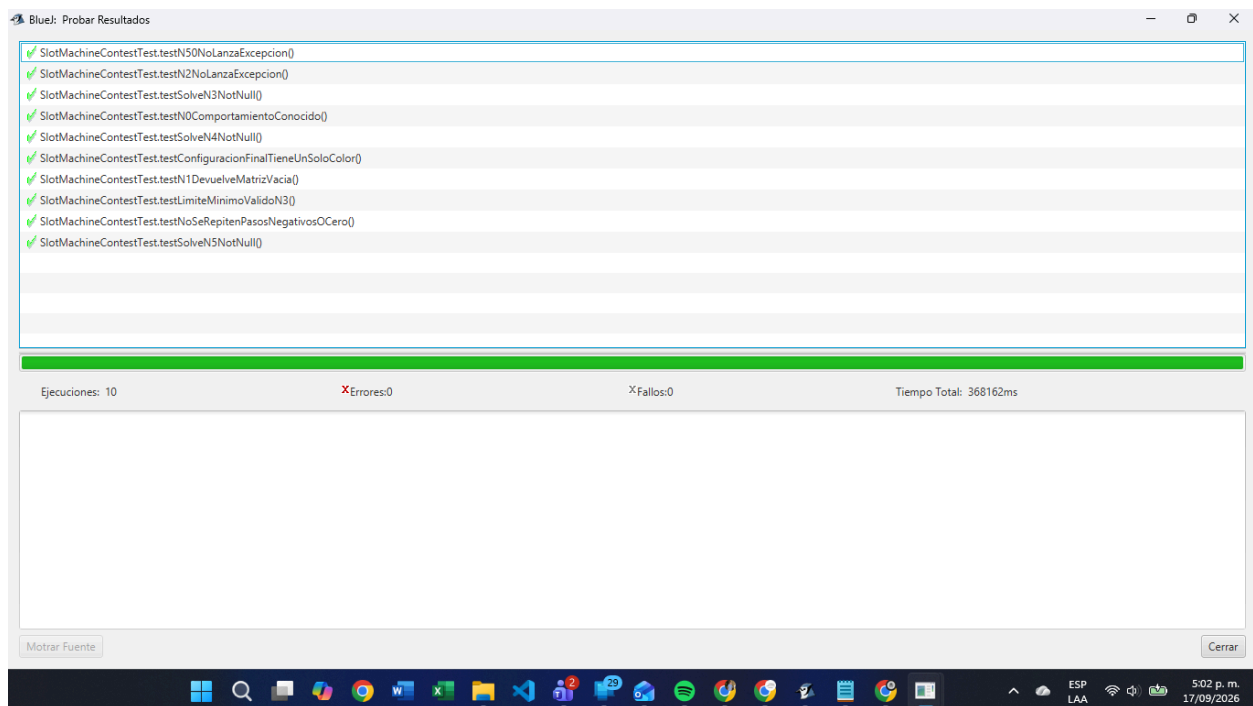
MINI-CICLO 5: Diseño y Cierre de Ciclo 3

Estado : Completo

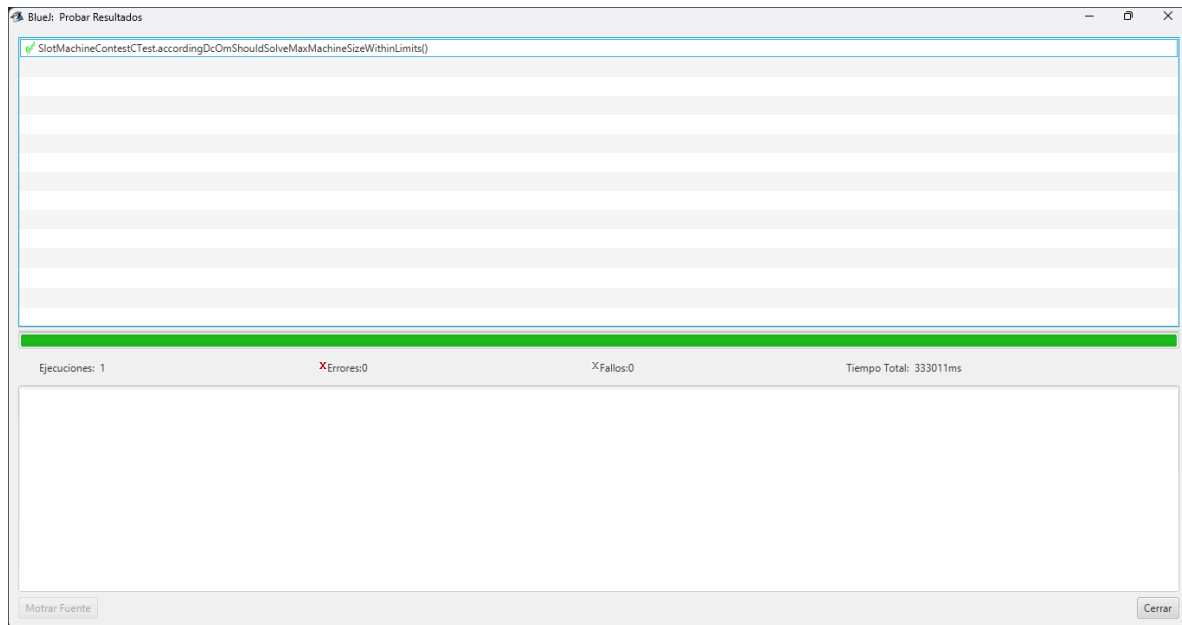
Objetivo: Diseñar los respectivos cambios que el astah debe soportar con las implementaciones anteriores.

- Actualizar el diseño completo en la herramienta Astah incorporando la clase SlotMachineContest y sus relaciones.
- Validar la implementación bajo los estándares de codificación y documentación de Java.
- Redactar el documento final de entrega integrando las partes correspondientes y la retrospectiva acumulativa.

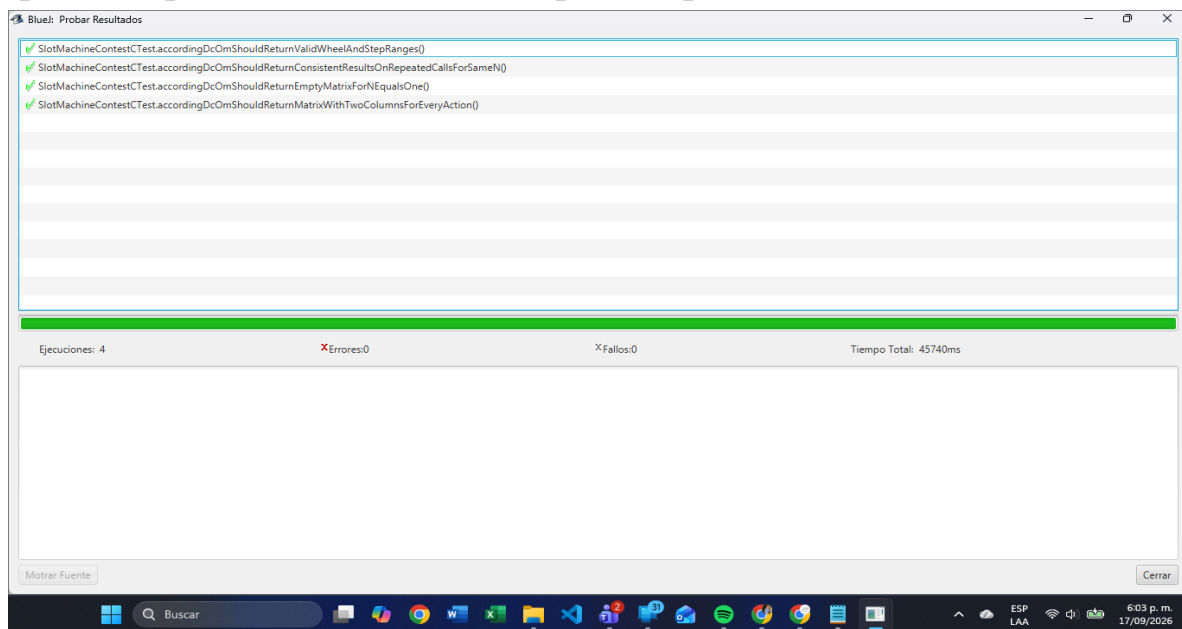
Captura de pantalla (Pruebas unitarias para SlotMachineContestTest)



Captura de pantalla (Prueba de aceptación 50 ruedas insertadas)



Captura de pantalla (Pruebas de aceptación para SlotMachineContestCTest)



2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?

El estado actual es 100% completado. Se implementaron y validaron la totalidad de los mini-ciclos previstos para el Ciclo 3

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

Invertimos aproximadamente 11 a 13 horas/hombre cada uno.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

El mayor logro fue la separación estricta de responsabilidades entre el motor de solución algorítmica y el simulador gráfico. Logramos que solve(n) actúe como un cliente autónomo que utiliza a SlotMachine exclusivamente como *testing tool* en modo invisible (respetando la restricción de diseño de interactuar únicamente mediante SlotMachine(n), spin(wheel, steps) y distinctSymbols()), produciendo una solución eficiente que puede ser ejecutada de forma desacoplada o visualizada secuencialmente mediante simulate(n).

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

- Problema: Encontrar una estrategia determinista para resolver el estado de la máquina utilizando únicamente la sonda de consulta distinctSymbols() sin romper el encapsulamiento para inspeccionar directamente los colores internos. Además, la ejecución de solve corría el riesgo de volverse lenta si se invocaban accidentalmente métodos con impacto gráfico o animaciones.
- Solución: Diseñamos un algoritmo iterativo que toma la primera rueda como referencia fija y rota las demás de manera sistemática evaluando las variaciones en el número de símbolos distintos (distinctSymbols()). Para garantizar un alto rendimiento, aseguramos que la máquina opere de forma 100% silenciosa e invisible durante solve, dejando las invocaciones de makeVisible() y pausas exclusivamente para simulate.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

- Lo que hicimos bien: Mantuvimos una comunicación constante en la división de los mini-ciclos y aplicamos un diseño guiado por contratos UML, respetando estrictamente las restricciones de los métodos que podían ser invocados por SlotMachineContest.
- Compromiso de mejora: Nos comprometemos a planificar con mayor anticipación los casos de prueba de aceptación extremos (como $n = 50$) desde el inicio del diseño algorítmico, asegurando que la complejidad computacional y el número de giros se mantengan óptimos desde las primeras iteraciones.

7. Considerando las prácticas XP incluidas en los laboratorios, ¿cuál fue la más útil? ¿Por qué?

La práctica más útil fue Diseño Simple y Entregas Pequeñas (Small Releases). Al abordar primero la construcción del constructor dinámico SlotMachine(n), luego la lógica matemática abstracta de solve, y finalmente la capa visual de simulate, evitamos la sobrecomplejidad y pudimos aislar y corregir los posibles fallos del algoritmo antes de intentar animarlo en la pantalla.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

- Escuela Colombiana de Ingeniería Julio Garavito. (2026). *Desarrollo Orientado Por Objetos [DOPO-POOB] Proyecto Inicial Ciclo No. 3/4 — Refactoring y Extensión*. Documento de requisitos, restricciones de diseño y diagrama UML. (La más útil, ya que definió los contratos estrictos entre SlotMachine y SlotMachineContest y las reglas del uso como testing tool).
- ICPC. (2025). *Problem I: Slot Machine*. International Collegiate Programming Contest.
- Escuela Colombiana de Ingeniería Julio Garavito. (2026). *DOPO - Instructivo de Pruebas Unitarias y Colectivas*. Guía para la convención de nomenclatura (accordingDcOmShould...).
- JUnit Team. (2025). *JUnit 4 Documentation*. Framework para pruebas unitarias automatizadas.